
EMPLOYMENT MANAGEMENT SYSTEM

The Security Model

Roles, the endpoint permission matrix, and the weak spots

AUTHORIZATION RULES • 2 OCTOBER 2026

Contents

01	1. The three roles	3
02	2. Where roles come from	3
03	3. The two layers of enforcement	4
04	4. Complete endpoint permission matrix	4
05	5. The ownership check in full	6
06	6. Token handling: what is strong and what is not	6
07	7. Error responses are safe by design	7
08	8. How the frontend mirrors the model	8
09	9. What to say if asked "is this app secure?"	8

One page that answers "who can do what, and where is that decided?"

1. The three roles

Role	Stored as	Meaning in this app
Admin	<code>ROLE_ADMIN</code>	Full control. Identical to MANAGER in behaviour.
Manager	<code>ROLE_MANAGER</code>	Full control. Identical to ADMIN in behaviour.
Employee	<code>ROLE_EMPLOYEE</code>	Own attendance only.

The `ROLE_` prefix is **not cosmetic** - Spring Security's `hasRole('X')` internally compares against `ROLE_X`, which is why the code says `hasRole('EMPLOYEE')` while the database says `ROLE_EMPLOYEE`. Writing `hasRole('ROLE_EMPLOYEE')` would silently never match.

`ADMIN` and `MANAGER` are separated only for future differentiation (audit logs, deletion rights). Today every check is `hasAnyRole('ADMIN', 'MANAGER')`.

2. Where roles come from

```
DataSeeder          → inserts ROLE_ADMIN / ROLE_MANAGER / ROLE_EMPLOYEE into `roles`
RoleRepository.findByName
UserRegistrationFacade → ALWAYS ROLE_EMPLOYEE
User.roles (join table) → the account's actual roles
```

There is **no endpoint, no UI and no code path that assigns ADMIN or MANAGER**. A fresh database has the three role rows but **zero user rows**, so the only way to obtain a manager account is to insert one manually (SQL, see `08-errors-and-glossary.md`) or to promote an existing account in the `users_roles` table.

That is why `hasAnyRole('ADMIN', 'MANAGER')` is effectively "is not an employee".

3. The two layers of enforcement

Layer 1 - URL level, SecurityConfig.java:34-37

```
.requestMatchers("/api/auth/**", "/error").permitAll()
.anyRequest().authenticated()
```

JAVA

Answers only: *is there a valid token?* It knows nothing about roles.

Note `/api/auth/**` is permitted, which includes `/api/auth/me`. That does **not** make `/me` public: the controller requires an `Authentication` argument, so an anonymous request still gets `401`. `permitAll` just means the filter chain does not reject it early.

Layer 2 - method level, @PreAuthorize

Answers: *does this token have a role that allows this?* Three variants are used:

Annotation	Where
<code>@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")</code> on the class	<code>EmployeeController</code> , <code>DepartmentController</code>
<code>@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")</code> on the method	<code>AttendanceController.search</code>
<code>@PreAuthorize("hasRole('EMPLOYEE')")</code>	<code>AttendanceController.me</code>
SpEL ownership check	<code>AttendanceController.checkIn/checkOut</code>

`@EnableMethodSecurity` in `SecurityConfig:22` is what activates these annotations; without it they would be silently ignored.

Class-level is preferred because it is impossible to forget on a new method. The attendance controller uses per-method annotations because it must mix roles.

Layer 3 - the frontend, usePermissions

Hides links, hides buttons, redirects pages, disables queries. **Convenience only.** Nothing here is a security control: deleting every `canManage` check would still leave the backend enforcing everything.

4. Complete endpoint permission matrix

`A/M` = ADMIN or MANAGER, `E` = EMPLOYEE. Anything not listed is not a real endpoint.

#	Method	Path	Role	Enforced by
1	POST	/api/auth/login	public	permitAll
2	POST	/api/auth/register	public	permitAll
3	GET	/api/auth/me	any authenticated	permitAll + controller arg
4	POST	/api/employees	A/M	class @PreAuthorize
5	GET	/api/employees	A/M	class
6	GET	/api/employees/{id}	A/M	class
7	GET	/api/employees/paged	A/M	class
8	GET	/api/employees/search	A/M	class
9	PATCH	/api/employees/{id}	A/M	class
10	DELETE	/api/employees/{id}	A/M	class
11	POST	/api/departments	A/M	class
12	GET	/api/departments	A/M	class
13	GET	/api/departments/paginated	A/M	class
14	GET	/api/departments/{id}	A/M	class
15	GET	/api/departments/{id}/employees	A/M	class
16	PUT	/api/departments/{id}	A/M	class
17	DELETE	/api/departments/{id}	A/M	class
18	POST	/api/attendance/search	A/M	method
19	GET	/api/attendance/me	E	method
20	POST	/api/attendance/check-in	A/M any, E self	SpEL
21	POST	/api/attendance/check-out	A/M any, E self	SpEL

What an EMPLOYEE gets, verified against a running server

Attempt	Result
GET /api/attendance/me	200 , only their own rows (17 in the test DB)
POST /api/attendance/search	403
GET /api/employees	403
GET /api/departments	403

Attempt	Result
POST /api/attendance/check-in?employeeId=<someone else>	403
POST /api/attendance/check-in?employeeId=<self>	200

5. The ownership check in full

```
@PreAuthorize("hasAnyRole('ADMIN','MANAGER') or " +
    "hasRole('EMPLOYEE') and " +
    "authentication.principal.employee ≠ null and " +
    "#employeeId == authentication.principal.employee.id")
```

JAVA

Token	Meaning
<code>hasAnyRole('ADMIN','MANAGER')</code>	managers may act on anyone
<code>#employeeId</code>	the method parameter, injected from the request
<code>authentication.principal</code>	the <code>User</code> entity placed in the context by the JWT filter
<code>authentication.principal.employee</code>	the linked <code>Employee</code> , or <code>null</code>
<code>authentication.principal.employee.id</code>	the id the token proves you own

`and` binds tighter than `or`, so this reads `(ADMIN or MANAGER) or (EMPLOYEE and employee ≠ null and #employeeId == ownId)`.

The `employee ≠ null` clause is what stops a manager-only account (no employee row) that somehow also had `ROLE_EMPLOYEE` from causing a null dereference inside the SpEL evaluation.

The same idea, done the safe way, appears on `/me`: the employee id is **not a parameter at all** - it is read from `user.getEmployee().getId()`. Not accepting it is stronger than validating it.

6. Token handling: what is strong and what is not

Strong

- Passwords are BCrypt-hashed (`BCryptPasswordEncoder`); the raw password is never stored or logged.
- The signature is verified on every request; a tampered token fails the parse and the filter degrades to anonymous -> `401`.
- Roles are re-read from the database on **every** request by the JWT filter, so the token's own `roles` claim is never trusted. Revoking a role takes effect on the caller's next request.

- CSRF is disabled *correctly*: there are no cookies, so there is nothing for a third-party site to ride on. The token is attached manually by JS.
- `Stateless` sessions - no server-side session to hijack.
- `Logout` clears the token from the client and the query cache.

Weak / worth knowing

Issue	Consequence	Where to fix
<code>jwt.expiration: 86400000</code> (24h)	a stolen token is usable for a day	shorter TTL + refresh tokens
Logout does not revoke the token	a copied token stays valid until expiry	a denylist or short TTL
<code>jwt.secret</code> is committed in <code>application.yaml</code>	in production this key must be injected, not committed	environment variable / secret manager
<code>spring.datasource.password</code> committed	same problem	environment variable
<code>ddl-auto: update</code> in production	schema is mutated at startup by the app	Flyway/Liquibase migrations, <code>validate</code>
<code>/api/auth/register</code> is public	anyone with a known employee email can claim an account	invite codes, or admin-created accounts only
<code>/error</code> is <code>permitAll</code>	the error page is reachable without a token	acceptable; it renders no data
No rate limiting on <code>/login</code>	password guessing is unthrottled	add a limiter or a delay
<code>getAuthorities()</code> derives from a <code>Set</code>	multi-role ordering is not guaranteed by Spring, but <code>getCurrentUser</code> sorts for display	already handled where it matters

7. Error responses are safe by design

```

401 {"status":401,"message":"Unauthorized: invalid or missing token"}      SecurityConfig
entryPoint
403 {"status":403,"message":"Forbidden: insufficient permissions"}      SecurityConfig
accessDeniedHandler

```

Both are written by the filter chain as **hardcoded strings** - they never interpolate anything from the request, so an error message cannot become an information leak.

Method-level denials (`@PreAuthorize`) throw `AccessDeniedException`, which is handled by `GlobalExceptionHandler` and produces the same message, just through the exception handler instead of the handler. The frontend only reads `errorData.message`, so both paths look identical to the UI.

The generic `500` handler logs the stack trace server-side and returns `"An unexpected error occurred. Please try again later."` - **never** the exception message. That is why a `LazyInitializationException` or an NPE shows up in the UI as a generic message and only in the console as a stack trace.

`BadCredentialsException` maps to `401 "Invalid username or password"`, which does not reveal whether the username exists.

8. How the frontend mirrors the model

Backend rule	Frontend mirror
<code>/employees</code> , <code>/departments</code> , <code>/</code> need A/M	<code>links</code> array <code>canManage: true</code> + <code><Navigate to="/attendance"></code> in three pages + <code>enabled: canManage</code>
<code>/attendance/search</code> needs A/M	<code>canManage ? searchAttendance(...) : fetchMyAttendance(...)</code>
<code>/attendance/me</code> needs E	the same ternary; a manager never calls <code>/me</code>
employee may only clock self	<code>effectiveClockEmployeeId</code> derived from <code>user.employeeId</code> ; the server double-checks

Because both sides read the same source of truth (`user.roles` from `/api/auth/me`), the UI cannot disagree with the API. If a role changes, the next `loadUser()` updates `canManage` and the UI follows.

9. What to say if asked "is this app secure?"

A defensible answer:

*Authentication is JWT-based, stateless, with BCrypt password hashing and per-request database role resolution. Authorization is enforced **server-side**, with class-level `@PreAuthorize` on the employee and department controllers, a dedicated `/attendance/me` read endpoint for employees, and a SpEL ownership check on clock-in/out. The frontend only hides UI. The remaining gaps are production configuration (committed secrets, `ddl-auto: update`, no migrations), the absence of token revocation and refresh, and a public registration endpoint that is mitigated by requiring a matching employee email.*